

REMIO

Plataforma de Serviços Domésticos

ENGENHARIA DE DADOS

Banco transacional, Data Warehouse e ETL

PostgreSQL · Modelo Dimensional · Pentaho Data Integration

Trabalho de Conclusão de Curso

MBA em Gestão Analítica com BI e Big Data

Universidade de Fortaleza — UNIFOR

Fortaleza — CE, 2026

SUMÁRIO

1 Visão Geral da Arquitetura de Dados.....	3
2 Banco de Dados Transacional (OLTP).....	4
2.1 Modelo de Dados (DER).....	4
2.2 Dicionário de Dados.....	6
3 Data Warehouse (Modelo Dimensional).....	8
3.1 Esquema Estrela.....	8
3.2 Dimensões.....	10
3.3 Tabelas de Fato.....	10
4 Processo de ETL no Pentaho (PDI).....	11
4.1 Job Principal de Carga.....	11
4.2 Anatomia de uma Transformation.....	11
4.3 Transformations e Estratégia de Carga.....	12
5 Do Dado ao Indicador — Mapeamento para os Dashboards.....	13
6 Evolução para Big Data.....	14

1 VISÃO GERAL DA ARQUITETURA DE DADOS

A engenharia de dados da plataforma Remio adota a arquitetura clássica de Business Intelligence, separando o ambiente operacional (onde a aplicação grava as transações do dia a dia) do ambiente analítico (otimizado para consultas e indicadores). O fluxo conecta cinco componentes, do site até os dashboards.

Figura 1 — Arquitetura geral de dados: do aplicativo aos dashboards de BI.



Fonte: elaborado pelos autores (2026).

O percurso do dado acontece em quatro etapas:

- **Captura (OLTP):** a aplicação grava cada ação — cadastro, pedido, orçamento, pagamento e avaliação — no banco transacional PostgreSQL, modelado de forma normalizada para garantir integridade.
- **Extração e transformação (ETL):** o Pentaho Data Integration (PDI) lê os dados do OLTP, aplica limpezas, conversões e o cálculo das chaves substitutas, e os organiza no formato dimensional.
- **Armazenamento analítico (DW):** o Data Warehouse, também em PostgreSQL, guarda os dados em um modelo estrela (dimensões e fatos), próprio para agregações rápidas.
- **Visualização (BI):** os três dashboards (cliente, prestador e empresa parceira) consultam o DW e exibem os indicadores.

Por que separar OLTP e DW? O banco transacional é otimizado para escritas pequenas e frequentes (muitos pedidos por segundo); o Data Warehouse é otimizado para leituras analíticas pesadas (somar o faturamento de meses). Misturar os dois degradaria tanto a operação quanto a análise — por isso a separação é uma boa prática consagrada em BI.

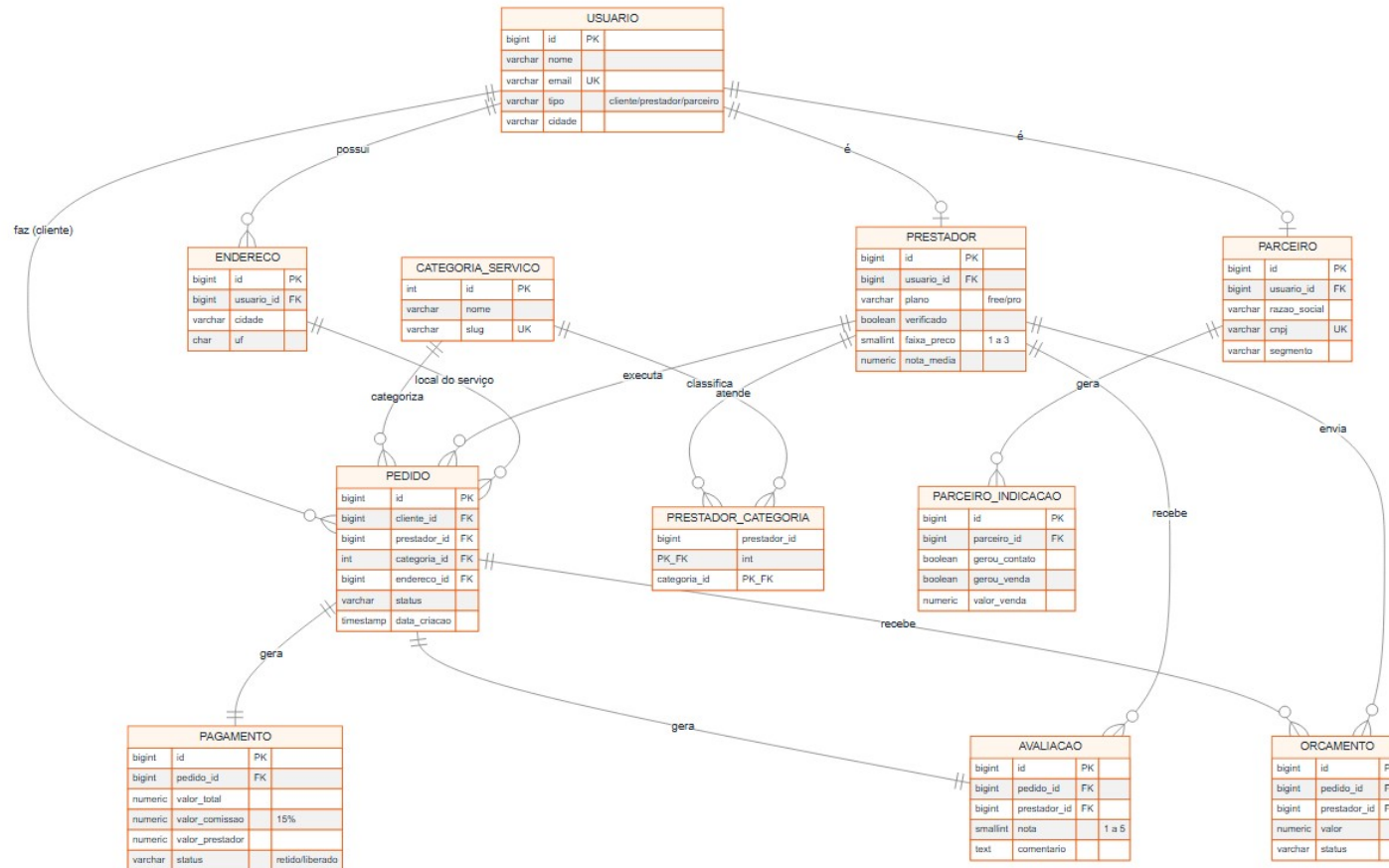
2 BANCO DE DADOS TRANSACIONAL (OLTP)

O banco transacional (sigla OLTP, de Online Transaction Processing) sustenta a operação da plataforma. Foi modelado de forma normalizada — sem redundância — em 11 tabelas que representam usuários, prestadores, parceiros, pedidos, orçamentos, pagamentos, avaliações e indicações.

2.1 Modelo de Dados (DER)

O Diagrama Entidade-Relacionamento (DER) — apresentado na página a seguir, em formato ampliado — mostra as tabelas e seus relacionamentos. A entidade central é o pedido, que conecta o cliente, o prestador, a categoria e o endereço, e do qual derivam o orçamento, o pagamento e a avaliação.

Figura 2 — Diagrama Entidade-Relacionamento do banco transacional (OLTP).



Fonte: elaborado pelos autores (2026).

A tabela a seguir resume a finalidade de cada entidade:

Tabela	Finalidade
usuario	Conta base de qualquer perfil (cliente, prestador ou parceiro).
categoria_servico	Tipos de serviço doméstico (faxina, elétrica, hidráulica, etc.).
prestador	Dados profissionais do prestador (plano, verificação, faixa de preço).
prestador_categoria	Relação N:N entre prestador e as categorias que ele atende.
parceiro	Empresas parceiras (lojas e fornecedores).
endereço	Endereços dos usuários (local de execução do serviço).
pedido	Solicitação de serviço — entidade central da operação.
orcamento	Propostas de valor enviadas pelos prestadores.
pagamento	Pagamento Protegido (custódia), com a comissão de 15%.
avaliacao	Nota e comentário do cliente após o serviço.
parceiro_indicacao	Indicações geradas para os parceiros.

2.2 Dicionário de Dados

Detalhamento das colunas das tabelas centrais da operação. O script completo, com todas as 11 tabelas, índices e restrições, está no arquivo 01-oltp-schema.sql do repositório.

usuario

Coluna	Tipo	Descrição
id	BIGSERIAL	Identificador único (chave primária).
nome	VARCHAR(150)	Nome completo do usuário.
email	VARCHAR(150)	E-mail de login (único).
senha_hash	VARCHAR(255)	Hash da senha (nunca em texto puro).
tipo	VARCHAR(20)	Papel: cliente, prestador ou parceiro.
cidade / uf	VARCHAR / CHAR(2)	Localização do usuário.
data_cadastro	TIMESTAMP	Data e hora de criação da conta.
ativo	BOOLEAN	Indica se a conta está ativa.

prestador

Coluna	Tipo	Descrição
id	BIGSERIAL	Identificador único.
usuario_id	BIGINT (FK)	Referência ao usuário (1:1).
plano	VARCHAR(10)	free ou pro (destaque na busca).
verificado	BOOLEAN	Selo de profissional verificado.
faixa_preco	SMALLINT	Faixa de preço: 1=\$, 2=\$\$, 3=\$\$\$.
preco_medio	NUMERIC(10,2)	Preço médio praticado.
nota_media	NUMERIC(3,2)	Média das avaliações recebidas.
total_avaliacoes	INTEGER	Quantidade de avaliações.

pedido

Coluna	Tipo	Descrição
id	BIGSERIAL	Identificador único.
cliente_id	BIGINT (FK)	Cliente que abriu o pedido.
prestador_id	BIGINT (FK)	Prestador que executa (pode ser nulo no início).
categoria_id	INT (FK)	Categoria do serviço.
endereço_id	BIGINT (FK)	Endereço de execução.
titulo	VARCHAR(150)	Título do pedido.
status	VARCHAR(20)	aguardando, orçamento, agendado, concluído, avaliado, cancelado.
data_criacao	TIMESTAMP	Abertura do pedido.
data_conclusao	TIMESTAMP	Conclusão do serviço (usada no ETL incremental).

pagamento

Coluna	Tipo	Descrição
id	BIGSERIAL	Identificador único.
pedido_id	BIGINT (FK)	Pedido associado (1:1).
valor_total	NUMERIC(10,2)	Valor pago pelo cliente.
percentual_comissao	NUMERIC(5,2)	Percentual da plataforma (padrão 15%).
valor_comissao	NUMERIC(10,2)	Comissão retida pela Remio.
valor_prestador	NUMERIC(10,2)	Valor repassado ao prestador.
metodo	VARCHAR(20)	pix, credito ou debito.
status	VARCHAR(20)	retido (custódia), liberado ou reembolsado.
data_liberacao	TIMESTAMP	Quando o valor foi liberado ao prestador.

avaliacao

Coluna	Tipo	Descrição
id	BIGSERIAL	Identificador único.
pedido_id	BIGINT (FK)	Pedido avaliado (1:1).
cliente_id	BIGINT (FK)	Cliente que avaliou.
prestador_id	BIGINT (FK)	Prestador avaliado.
nota	SMALLINT	Nota de 1 a 5.
comentario	TEXT	Comentário livre.
data	TIMESTAMP	Data da avaliação.

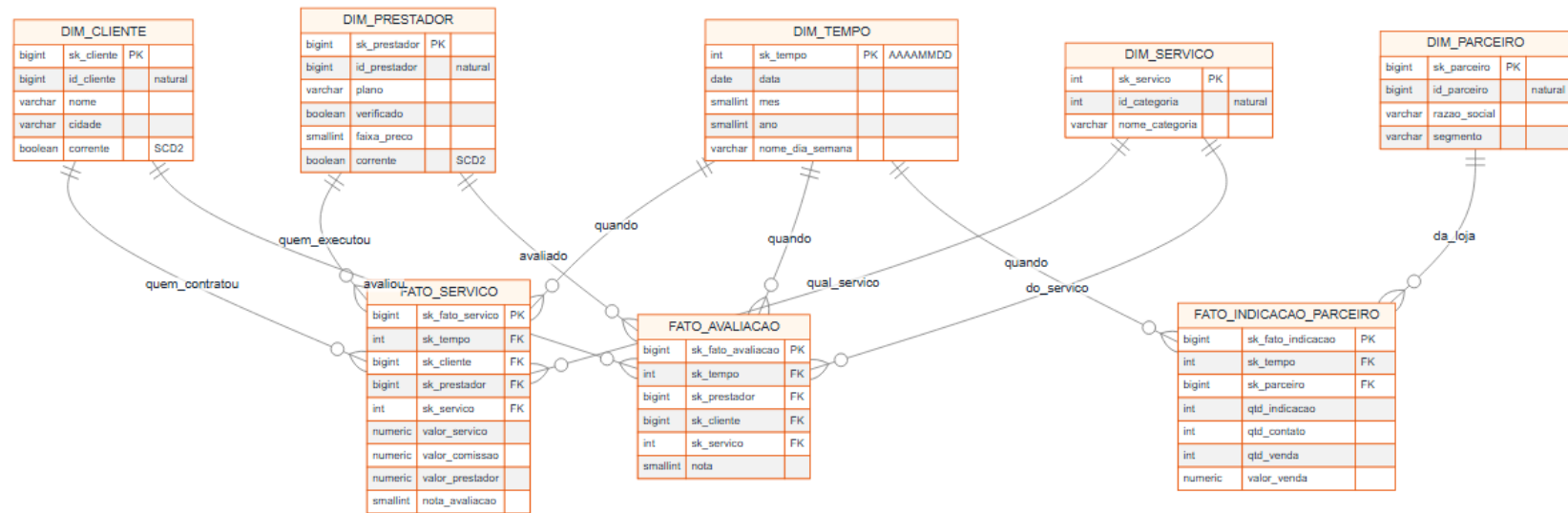
3 DATA WAREHOUSE (MODELO DIMENSIONAL)

O Data Warehouse (DW) é o ambiente analítico, modelado segundo a técnica dimensional de Ralph Kimball. Em vez de tabelas normalizadas, organiza-se em tabelas de dimensão (o contexto: quem, o quê, quando) e tabelas de fato (as métricas: quanto, quantos), formando um esquema estrela.

3.1 Esquema Estrela

A Remio possui três tabelas de fato que compartilham dimensões comuns — uma configuração chamada constelação de fatos. As cinco dimensões (tempo, cliente, prestador, serviço e parceiro) são reaproveitadas pelos fatos, garantindo consistência analítica. O esquema é apresentado na página a seguir, em formato ampliado.

Figura 3 — Esquema estrela do Data Warehouse (constelação de fatos).



Fonte: elaborado pelos autores (2026).

Convenção de chaves: cada dimensão tem uma chave substituta (surrogate key, prefixo sk_) gerada no próprio DW, além da chave natural (prefixo id_) herdada do OLTP. As chaves substitutas tornam as junções mais rápidas e permitem o versionamento histórico das dimensões.

3.2 Dimensões

Dimensão	Conteúdo	Tipo SCD
dim_tempo	Calendário com chave AAAAMMDD; dia, mês, ano, trimestre, dia da semana.	Carga única
dim_cliente	Dados do cliente (nome, cidade) com histórico de mudanças.	SCD Tipo 2
dim_prestador	Dados do prestador (plano, verificação, faixa de preço) com histórico.	SCD Tipo 2
dim_servico	Categoria do serviço.	SCD Tipo 1
dim_parceiro	Empresa parceira (razão social, segmento, cidade).	SCD Tipo 1

SCD (Slowly Changing Dimension): estratégia para lidar com mudanças nos dados de uma dimensão. No Tipo 1, o valor é simplesmente sobrescrito; no Tipo 2, cria-se uma nova versão da linha (com datas de início e fim e um indicador de versão corrente), preservando o histórico — essencial, por exemplo, para saber qual era o plano do prestador na época de cada serviço.

3.3 Tabelas de Fato

Fato	Grão (1 linha por...)	Principais medidas
fato_servico	serviço concluído (pedido)	valor_servico, valor_comissao, valor_prestador, qtd, nota
fato_avaliacao	avaliação registrada	nota, qtd_avaliacao
fato_indicacao_parceiro	indicação de parceiro	qtd_indicacao, qtd_contato, qtd_venda, valor_venda

fato_servico (principal)

Coluna	Tipo	Descrição
sk_fato_servico	BIGSERIAL	Chave primária do fato.
sk_tempo	INTEGER (FK)	Quando o serviço foi concluído.
sk_cliente	BIGINT (FK)	Quem contratou.
sk_prestador	BIGINT (FK)	Quem executou.
sk_servico	INTEGER (FK)	Categoria do serviço.
id_pedido	BIGINT	Chave degenerada (rastreo ao pedido de origem).
valor_servico	NUMERIC(10,2)	Valor total do serviço.
valor_comissao	NUMERIC(10,2)	Comissão da plataforma.
valor_prestador	NUMERIC(10,2)	Valor recebido pelo prestador.
nota_avaliacao	SMALLINT	Nota recebida (se avaliado).

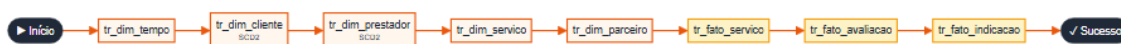
4 PROCESSO DE ETL NO PENTAHO (PDI)

O ETL (Extract, Transform, Load) é o processo que move e adapta os dados do OLTP para o DW. Foi projetado no Pentaho Data Integration (PDI, também conhecido como Kettle), que organiza o trabalho em dois tipos de artefato: transformations (.ktr), que manipulam os dados linha a linha, e jobs (.kjb), que orquestram a ordem de execução das transformations.

4.1 Job Principal de Carga

O job `job_carga_dw.kjb` coordena toda a carga. A ordem respeita a dependência referencial: primeiro carregam-se as dimensões, depois os fatos — pois os fatos precisam das chaves substitutas (`sk_`) já existentes nas dimensões para fazer seus lookups.

Figura 4 — Job principal: carga das dimensões seguida da carga dos fatos.

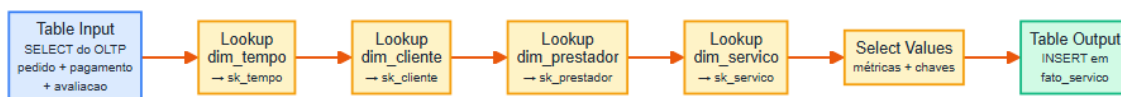


Fonte: elaborado pelos autores (2026).

4.2 Anatomia de uma Transformation

Cada transformation é uma sequência de steps (passos) ligados por hops (setas) por onde fluem as linhas. O exemplo abaixo detalha a `tr_fato_servico`, que carrega a tabela de fato principal.

Figura 5 — Steps da transformation `tr_fato_servico`, do OLTP ao fato.



Fonte: elaborado pelos autores (2026).

Os steps dessa transformation:

Step (PDI)	Função
Table Input	Lê do OLTP os pedidos concluídos, unindo pedido, pagamento e avaliacao via SQL.
Lookup dim_tempo	Converte a data de conclusão na chave <code>sk_tempo</code> (formato AAAAMMDD).
Lookup dim_cliente	Busca a <code>sk_cliente</code> vigente a partir do id natural do cliente.

Step (PDI)	Função
Lookup dim_prestador	Busca a sk_prestador vigente a partir do id natural do prestador.
Lookup dim_servico	Busca a sk_servico a partir da categoria.
Select Values	Seleciona e ordena as colunas finais (chaves + medidas).
Table Output	Insere as linhas resultantes na tabela fato_servico do DW.

4.3 Transformations e Estratégia de Carga

Transformation	Origem (OLTP)	Destino (DW)	Carga
tr_dim_tempo	Geração por intervalo	dim_tempo	Única
tr_dim_cliente	usuario (cliente)	dim_cliente	SCD2
tr_dim_prestador	prestador + usuario	dim_prestador	SCD2
tr_dim_servico	categoria_servico	dim_servico	SCD1
tr_dim_parceiro	parceiro + usuario	dim_parceiro	SCD1
tr_fato_servico	pedido + pagamento + avaliacao	fato_servico	Incremental
tr_fato_avaliacao	avaliacao	fato_avaliacao	Incremental
tr_fato_indicacao	parceiro_indicacao	fato_indicacao_parceiro	Incremental

Carga incremental: para não reprocessar toda a base a cada execução, os fatos são carregados apenas com os registros novos desde a última carga. Guarda-se a data/hora da última execução bem-sucedida em uma tabela de controle; a Table Input filtra por data_conclusao maior que esse marco. Isso torna a carga rápida e adequada a uma execução diária ou de hora em hora.

Dimensões SCD2 no Pentaho: o step nativo Dimension Lookup/Update do PDI cuida automaticamente do versionamento — ele detecta mudanças, encerra a versão anterior (preenchendo dt_fim e marcando corrente = false) e cria a nova versão vigente, sem necessidade de código manual.

5 DO DADO AO INDICADOR — MAPEAMENTO PARA OS DASHBOARDS

A tabela a seguir fecha o ciclo, mostrando como cada indicador exibido nos dashboards do protótipo é obtido a partir do Data Warehouse. Isso evidencia que a modelagem foi desenhada de trás para frente: a partir das perguntas de negócio que os painéis respondem.

Dashboard / Indicador	Fato	Dimensões	Cálculo
Cliente — Gastos por mês	fato_servico	dim_tempo, dim_cliente	SUM(valor_servico)
Cliente — Contratações por categoria	fato_servico	dim_servico, dim_cliente	COUNT(*)
Prestador — Faturamento mensal	fato_servico	dim_tempo, dim_prestador	SUM(valor_prestador)
Prestador — Demanda por serviço	fato_servico	dim_servico, dim_prestador	COUNT(*)
Prestador — Nota média	fato_avaliacao	dim_prestador	AVG(nota)
Prestador — Preço vs. média	fato_servico	dim_servico, dim_prestador	AVG(valor_servico)
Parceiro — Funil indicação→venda	fato_indicacao_parceiro	dim_tempo, dim_parceiro	SUM(qtd_indicacao/contato/venda)

Um exemplo concreto da consulta que alimenta o faturamento mensal do prestador:

```
SELECT t.ano, t.mes, t.nome_mes,
       SUM(f.valor_prestador) AS faturamento
FROM   fato_servico f
JOIN   dim_tempo    t ON t.sk_tempo    = f.sk_tempo
JOIN   dim_prestador p ON p.sk_prestador = f.sk_prestador
WHERE  p.id_prestador = :id AND p.corrente = TRUE
GROUP BY t.ano, t.mes, t.nome_mes
ORDER BY t.ano, t.mes;
```

6 EVOLUÇÃO PARA BIG DATA

A arquitetura apresentada atende plenamente a operação inicial da Remio. À medida que o volume de dados crescer — milhões de pedidos, eventos de navegação, geolocalização e mensagens — alguns componentes podem evoluir para um ecossistema de Big Data, mantendo o mesmo princípio de separar operação e análise:

- **Ingestão de eventos em tempo real:** uma camada de streaming (por exemplo, Apache Kafka) capturaria eventos do aplicativo de forma contínua, complementando a carga em lote do Pentaho.
- **Data Lake:** dados brutos e semiestruturados (logs, cliques, avaliações em texto) seriam armazenados em um data lake (por exemplo, em formato Parquet sobre armazenamento de objetos) antes do tratamento.
- **Processamento distribuído:** transformações sobre grandes volumes poderiam usar Apache Spark, preservando a lógica dimensional já definida.
- **Analytics avançado:** com a base histórica consolidada, tornam-se viáveis modelos preditivos — previsão de demanda por região, recomendação de prestadores e detecção de fraude no Pagamento Protegido.

Em todos os cenários, o modelo dimensional documentado aqui permanece como a camada de consumo analítico — a fonte única e confiável que alimenta os dashboards e as decisões de negócio da plataforma.